

10 minutes to pandas

Intro to data structures

Essential basic functionality

IO tools (text, CSV, HDF5, ...)

PyArrow Functionality

Indexing and selecting data

MultiIndex / advanced indexing

Copy-on-Write (CoW)

Merge, join, concatenate and compare

Reshaping and pivot tables

Working with text data

Working with missing data

Duplicate Labels

Categorical data

Nullable integer data type

Nullable Boolean data type

Chart visualization

Table Visualization

User-Defined Functions (UDFs)

10 minutes to pandas

This is a short introduction to pandas, geared mainly for new users. You can see more complex recipes in the [Cookbook](#).

Customarily, we import as follows:

```
In [1]: import numpy as np
```

```
In [2]: import pandas as pd
```

Basic data structures in pandas

pandas provides two types of classes for handling data:

1. **[Series](#)**: a **one-dimensional labeled array holding data of any type**
such as integers, strings, Python objects etc.
2. **[DataFrame](#)**: a two-dimensional data structure that holds data like a two-dimension array or a table with rows and columns.

Object creation

See the [Intro to data structures section](#).

```
In [5]: dates = pd.date_range("20130101", periods=6)
```

```
In [6]: dates
```

```
Out[6]:
```

```
DatetimeIndex(['2013-01-01', '2013-01-02', '2013-01-03', '2013-01-04',  
               '2013-01-05', '2013-01-06'],  
              dtype='datetime64[us]', freq='D')
```

```
In [7]: df = pd.DataFrame(np.random.randn(6, 4), index=dates, columns=list("ABCD"))
```

```
In [8]: df
```

```
Out[8]:
```

	A	B	C	D
2013-01-01	0.469112	-0.282863	-1.509059	-1.135632
2013-01-02	1.212112	-0.173215	0.119209	-1.044236
2013-01-03	-0.861849	-2.104569	-0.494929	1.071804
2013-01-04	0.721555	-0.706771	-1.039575	0.271860
2013-01-05	-0.424972	0.567020	0.276232	-1.087401
2013-01-06	-0.673690	0.113648	-1.478427	0.524988

Creating a `DataFrame` by passing a dictionary of objects where the keys are the column labels and the values are the column values.

```
In [12]: df2.<TAB> # noqa: E225, E999
df2.A          df2.bool
df2.abs        df2.boxplot
df2.add        df2.C
df2.add_prefix df2.clip
df2.add_suffix df2.columns
df2.align      df2.copy
df2.all        df2.count
df2.any        df2.combine
df2.append     df2.D
df2.apply      df2.describe
df2.B          df2.duplicated
df2.diff
```

As you can see, the columns `A`, `B`, `C`, and `D` are automatically tab completed. `E` and `F` are there as well; the rest of the attributes have been truncated for brevity.

Viewing data

See the [Essential basic functionality section](#).

Use `DataFrame.head()` and `DataFrame.tail()` to view the top and bottom rows of the frame respectively:

```
In [17]: df.to_numpy()
Out[17]:
array([[ 0.4691, -0.2829, -1.5091, -1.1356],
       [ 1.2121, -0.1732,  0.1192, -1.0442],
       [-0.8618, -2.1046, -0.4949,  1.0718],
       [ 0.7216, -0.7068, -1.0396,  0.2719],
       [-0.425 ,  0.567 ,  0.2762, -1.0874],
       [-0.6737,  0.1136, -1.4784,  0.525 ]])
```

Note

NumPy arrays have one dtype for the entire array while pandas DataFrames have one dtype per column. When you call `DataFrame.to_numpy()`, pandas will find the NumPy dtype that can hold *all* of the dtypes in the DataFrame. If the common data type is `object`, `DataFrame.to_numpy()` will require copying data.

```
In [18]: df2.dtypes
Out[18]:
A          float64
B    datetime64[us]
C          float32
D          int32
E          category
F              str
dtype: object
```

```
In [19]: df2.to_numpy()
Out[19]:
array([[1.0, Timestamp('2013-01-02 00:00:00'), 1.0, 3, 'test', 'foo'],
       [1.0, Timestamp('2013-01-02 00:00:00'), 1.0, 3, 'train', 'foo'],
       [1.0, Timestamp('2013-01-02 00:00:00'), 1.0, 3, 'test', 'foo'],
       [1.0, Timestamp('2013-01-02 00:00:00'), 1.0, 3, 'train', 'foo']],
      dtype=object)
```

```
In [22]: df.sort_index(axis=1, ascending=False)
Out[22]:
```

	D	C	B	A
2013-01-01	-1.135632	-1.509059	-0.282863	0.469112
2013-01-02	-1.044236	0.119209	-0.173215	1.212112
2013-01-03	1.071804	-0.494929	-2.104569	-0.861849
2013-01-04	0.271860	-1.039575	-0.706771	0.721555
2013-01-05	-1.087401	0.276232	0.567020	-0.424972
2013-01-06	0.524988	-1.478427	0.113648	-0.673690

`DataFrame.sort_values()` sorts by values:

```
In [23]: df.sort_values(by="B")
Out[23]:
```

	A	B	C	D
2013-01-03	-0.861849	-2.104569	-0.494929	1.071804
2013-01-04	0.721555	-0.706771	-1.039575	0.271860
2013-01-01	0.469112	-0.282863	-1.509059	-1.135632
2013-01-02	1.212112	-0.173215	0.119209	-1.044236
2013-01-06	-0.673690	0.113648	-1.478427	0.524988
2013-01-05	-0.424972	0.567020	0.276232	-1.087401

Selection

Note

While standard Python / NumPy expressions for selecting and setting are intuitive and come in handy for interactive work, for production code, we recommend the optimized pandas data access methods, `DataFrame.at()`, `DataFrame.iat()`, `DataFrame.loc()` and `DataFrame.iloc()`.

```
In [26]: df[["B", "A"]]
Out[26]:
```

	B	A
2013-01-01	-0.282863	0.469112
2013-01-02	-0.173215	1.212112
2013-01-03	-2.104569	-0.861849
2013-01-04	-0.706771	0.721555
2013-01-05	0.567020	-0.424972
2013-01-06	0.113648	-0.673690

For a `DataFrame`, passing a slice `:` selects matching rows:

```
In [27]: df[0:3]
Out[27]:
```

	A	B	C	D
2013-01-01	0.469112	-0.282863	-1.509059	-1.135632
2013-01-02	1.212112	-0.173215	0.119209	-1.044236
2013-01-03	-0.861849	-2.104569	-0.494929	1.071804

```
In [28]: df["20130102":"20130104"]
Out[28]:
```

	A	B	C	D
2013-01-02	1.212112	-0.173215	0.119209	-1.044236
2013-01-03	-0.861849	-2.104569	-0.494929	1.071804
2013-01-04	0.721555	-0.706771	-1.039575	0.271860

Selection by label

See more in [Selection by Label](#) using `DataFrame.loc()` or `DataFrame.at()`.

Selecting a row matching a label:

```
In [32]: df.loc[dates[0], "A"]  
Out[32]: np.float64(0.4691122999071863)
```

For getting fast access to a scalar (equivalent to the prior method):

```
In [33]: df.at[dates[0], "A"]  
Out[33]: np.float64(0.4691122999071863)
```

Selection by position

See more in [Selection by Position](#) using `DataFrame.iloc()` or `DataFrame.iat()`.

Select via the position of the passed integers:

```
In [34]: df.iloc[3]  
Out[34]:  
A    0.721555  
B   -0.706771  
C   -1.039575  
D    0.271860  
Name: 2013-01-04 00:00:00, dtype: float64
```

Integer slices acts similar to NumPy/Python:

```
In [38]: df.iloc[:, 1:3]
Out[38]:
```

	B	C
2013-01-01	-0.282863	-1.509059
2013-01-02	-0.173215	0.119209
2013-01-03	-2.104569	-0.494929
2013-01-04	-0.706771	-1.039575
2013-01-05	0.567020	0.276232
2013-01-06	0.113648	-1.478427

For getting a value explicitly:

```
In [39]: df.iloc[1, 1]
Out[39]: np.float64(-0.17321464905330858)
```

For getting fast access to a scalar (equivalent to the prior method):

```
In [40]: df.iat[1, 1]
Out[40]: np.float64(-0.17321464905330858)
```

Boolean indexing

Select rows where `df.A` is greater than `0`.


```

In [43]: df2 = df.copy()

In [44]: df2["E"] = ["one", "one", "two", "three", "four", "three"]

In [45]: df2
Out[45]:

```

	A	B	C	D	E
2013-01-01	0.469112	-0.282863	-1.509059	-1.135632	one
2013-01-02	1.212112	-0.173215	0.119209	-1.044236	one
2013-01-03	-0.861849	-2.104569	-0.494929	1.071804	two
2013-01-04	0.721555	-0.706771	-1.039575	0.271860	three
2013-01-05	-0.424972	0.567020	0.276232	-1.087401	four
2013-01-06	-0.673690	0.113648	-1.478427	0.524988	three

```

In [46]: df2[df2["E"].isin(["two", "four"])]
Out[46]:

```

	A	B	C	D	E
2013-01-03	-0.861849	-2.104569	-0.494929	1.071804	two
2013-01-05	-0.424972	0.567020	0.276232	-1.087401	four

Setting

Setting a new column automatically aligns the data by the indexes:

```
In [53]: df
```

```
Out[53]:
```

	A	B	C	D	F
2013-01-01	0.000000	0.000000	-1.509059	5.0	NaN
2013-01-02	1.212112	-0.173215	0.119209	5.0	1.0
2013-01-03	-0.861849	-2.104569	-0.494929	5.0	2.0
2013-01-04	0.721555	-0.706771	-1.039575	5.0	3.0
2013-01-05	-0.424972	0.567020	0.276232	5.0	4.0
2013-01-06	-0.673690	0.113648	-1.478427	5.0	5.0

A `where` operation with setting:

```
In [54]: df2 = df.copy()
```

```
In [55]: df2[df2 > 0] = -df2
```

```
In [56]: df2
```

```
Out[56]:
```

	A	B	C	D	F
2013-01-01	0.000000	0.000000	-1.509059	-5.0	NaN
2013-01-02	-1.212112	-0.173215	-0.119209	-5.0	-1.0
2013-01-03	-0.861849	-2.104569	-0.494929	-5.0	-2.0
2013-01-04	-0.721555	-0.706771	-1.039575	-5.0	-3.0
2013-01-05	-0.424972	-0.567020	-0.276232	-5.0	-4.0
2013-01-06	-0.673690	-0.113648	-1.478427	-5.0	-5.0

Missing data

For NumPy data types, `np.nan` represents missing data. It is by default not included in computations. See the [Missing Data section](#).

Reindexing allows you to change/add/delete the index on a specified axis. This returns a copy of the data:

```
In [62]: pd.isna(df1)
```

```
Out[62]:
```

	A	B	C	D	F	E
2013-01-01	False	False	False	False	True	False
2013-01-02	False	False	False	False	False	False
2013-01-03	False	False	False	False	False	True
2013-01-04	False	False	False	False	False	True

Operations

See the [Basic section on Binary Ops.](#)

Stats

Operations in general *exclude* missing data.

Calculate the mean value for each column:

```
In [63]: df.mean()
```

```
Out[63]:
```

```
A    -0.004474
B    -0.383981
C    -0.687758
D     5.000000
F     3.000000
dtype: float64
```

Calculate the mean value for each row:

User defined functions

`DataFrame.agg()` and `DataFrame.transform()` applies a user defined function that reduces or broadcasts its result respectively.

```
In [68]: df.agg(lambda x: np.mean(x) * 5.6)
```

```
Out[68]:
```

```
A    -0.025054  
B    -2.150294  
C    -3.851445  
D     28.000000  
F     16.800000  
dtype: float64
```

```
In [69]: df.transform(lambda x: x * 101.2)
```

```
Out[69]:
```

	A	B	C	D	F
2013-01-01	0.000000	0.000000	-152.716721	506.0	NaN
2013-01-02	122.665737	-17.529322	12.063922	506.0	101.2
2013-01-03	-87.219115	-212.982405	-50.086843	506.0	202.4
2013-01-04	73.021382	-71.525239	-105.204988	506.0	303.6
2013-01-05	-43.007200	57.382459	27.954680	506.0	404.8
2013-01-06	-68.177398	11.501219	-149.616767	506.0	506.0

Value Counts

See more at [Histogramming and Discretization](#).

```
In [73]: s = pd.Series(["A", "B", "C", "Aaba", "Baca", np.nan, "CABA", "dog", "cat"])

In [74]: s.str.lower()
Out[74]:
0      a
1      b
2      c
3    aaba
4    baca
5     NaN
6    caba
7     dog
8     cat
dtype: str
```

Merge

Concat

pandas provides various facilities for easily combining together [Series](#) and [DataFrame](#) objects with various kinds of set logic for the indexes and relational algebra functionality in the case of join / merge-type operations.

See the [Merging section](#).

Concatenating pandas objects together row-wise with [concat\(\)](#):

Note

Adding a column to a `DataFrame` is relatively fast. However, adding a row requires a copy, and may be expensive. We recommend passing a pre-built list of records to the `DataFrame` constructor instead of building a `DataFrame` by iteratively appending records to it.

Join

`merge()` enables SQL style join types along specific columns. See the [Database style joining](#) section.

```
In [79]: left = pd.DataFrame({"key": ["foo", "foo"], "lval": [1, 2]})
```

```
In [80]: right = pd.DataFrame({"key": ["foo", "foo"], "rval": [4, 5]})
```

```
In [81]: left
```

```
Out[81]:
```

	key	lval
0	foo	1
1	foo	2

```
In [82]: right
```

```
Out[82]:
```

	key	rval
0	foo	4
1	foo	5

```
In [83]: pd.merge(left, right, on="key")
```

```
Out[83]:
```

	key	lval	rval
0	foo	1	4
1	foo	1	5
2	foo	2	4
3	foo	2	5

See the [Grouping section](#).

```
In [89]: df = pd.DataFrame(  
.....:     {  
.....:         "A": ["foo", "bar", "foo", "bar", "foo", "bar", "foo", "foo"],  
.....:         "B": ["one", "one", "two", "three", "two", "two", "one", "three"],  
.....:         "C": np.random.randn(8),  
.....:         "D": np.random.randn(8),  
.....:     }  
.....: )  
.....:
```

```
In [90]: df
```

```
Out[90]:
```

	A	B	C	D
0	foo	one	1.346061	-1.577585
1	bar	one	1.511763	0.396823
2	foo	two	1.627081	-0.105381
3	bar	three	-0.990582	-0.532532
4	foo	two	-0.441652	1.453749
5	bar	two	1.211526	1.208843
6	foo	one	0.268520	-0.080952
7	foo	three	0.024580	-0.264610

Grouping by a column label, selecting column labels, and then applying the `DataFrameGroupBy.sum()` function to the resulting groups:

```
In [91]: df.groupby("A")[["C", "D"]].sum()
```

```
Out[91]:
```

	C	D
A		
bar	1.732707	1.073134
foo	2.824590	-0.574779

Grouping by multiple columns label forms `MultiIndex`.

```

In [93]: arrays = [
....:     ["bar", "bar", "baz", "baz", "foo", "foo", "qux", "qux"],
....:     ["one", "two", "one", "two", "one", "two", "one", "two"],
....: ]
....:

In [94]: index = pd.MultiIndex.from_arrays(arrays, names=["first", "second"])

In [95]: df = pd.DataFrame(np.random.randn(8, 2), index=index, columns=["A", "B"])

In [96]: df2 = df[:4]

In [97]: df2
Out[97]:

```

		A	B
first	second		
bar	one	-0.727965	-0.589346
	two	0.339969	-0.693205
baz	one	-0.339355	0.593616
	two	0.884345	1.591431

The `stack()` method “compresses” a level in the DataFrame’s columns:


```
In [100]: stacked.unstack()
Out[100]:
```

		A	B
first	second		
bar	one	-0.727965	-0.589346
	two	0.339969	-0.693205
baz	one	-0.339355	0.593616
	two	0.884345	1.591431

```
In [101]: stacked.unstack(1)
Out[101]:
```

		one	two
second			
first			
bar	A	-0.727965	0.339969
	B	-0.589346	-0.693205
baz	A	-0.339355	0.884345
	B	0.593616	1.591431

```
In [102]: stacked.unstack(0)
Out[102]:
```

		bar	baz
first			
second			
one	A	-0.727965	-0.339355
	B	-0.589346	0.593616
two	A	0.339969	0.884345
	B	-0.693205	1.591431

Pivot tables

See the section on [Pivot Tables](#).

```
In [105]: pd.pivot_table(df, values="D", index=["A", "B"], columns=["C"])
Out[105]:
```

		bar	foo
A	B		
one	A	2.395985	-1.202872
	B	1.395433	-1.814470
	C	-0.392670	-0.055224
three	A	-0.595447	NaN
	B	NaN	1.928123
	C	0.166599	NaN
two	A	NaN	0.007207
	B	1.552825	NaN
	C	NaN	1.018601

Time series

pandas has simple, powerful, and efficient functionality for performing resampling operations during frequency conversion (e.g., converting secondly data into 5-minutely data). This is extremely common in, but not limited to, financial applications. See the [Time Series section](#).

```
In [106]: rng = pd.date_range("1/1/2012", periods=100, freq="s")

In [107]: ts = pd.Series(np.random.randint(0, 500, len(rng)), index=rng)

In [108]: ts.resample("5Min").sum()
Out[108]:
2012-01-01    24182
Freq: 5min, dtype: int64
```

`Series.tz_localize()` localizes a time series to a time zone:

```
In [115]: rng
Out[115]:
DatetimeIndex(['2012-03-06', '2012-03-07', '2012-03-08', '2012-03-09',
              '2012-03-10'],
              dtype='datetime64[us]', freq='D')

In [116]: rng + pd.offsets.BusinessDay(5)
Out[116]:
DatetimeIndex(['2012-03-13', '2012-03-14', '2012-03-15', '2012-03-16',
              '2012-03-16'],
              dtype='datetime64[us]', freq=None)
```

Categoricals

pandas can include categorical data in a `DataFrame`. For full docs, see the [categorical introduction](#) and the [API documentation](#).

```
In [117]: df = pd.DataFrame(
.....:     {"id": [1, 2, 3, 4, 5, 6], "raw_grade": ["a", "b", "b", "a", "a", "e"]}
.....: )
.....:
```

Converting the raw grades to a categorical data type:

```

In [122]: df["grade"] = df["grade"].cat.set_categories(
.....:     ["very bad", "bad", "medium", "good", "very good"]
.....: )
.....:

In [123]: df["grade"]
Out[123]:
0    very good
1         good
2         good
3    very good
4    very good
5    very bad
Name: grade, dtype: category
Categories (5, str): ['very bad', 'bad', 'medium', 'good', 'very good']

```

Sorting is per order in the categories, not lexical order:

```

In [124]: df.sort_values(by="grade")
Out[124]:
   id raw_grade  grade
5   6         e  very bad
1   2         b    good
2   3         b    good
0   1         a  very good
3   4         a  very good
4   5         a  very good

```

Grouping by a categorical column with `observed=False` also shows empty categories: